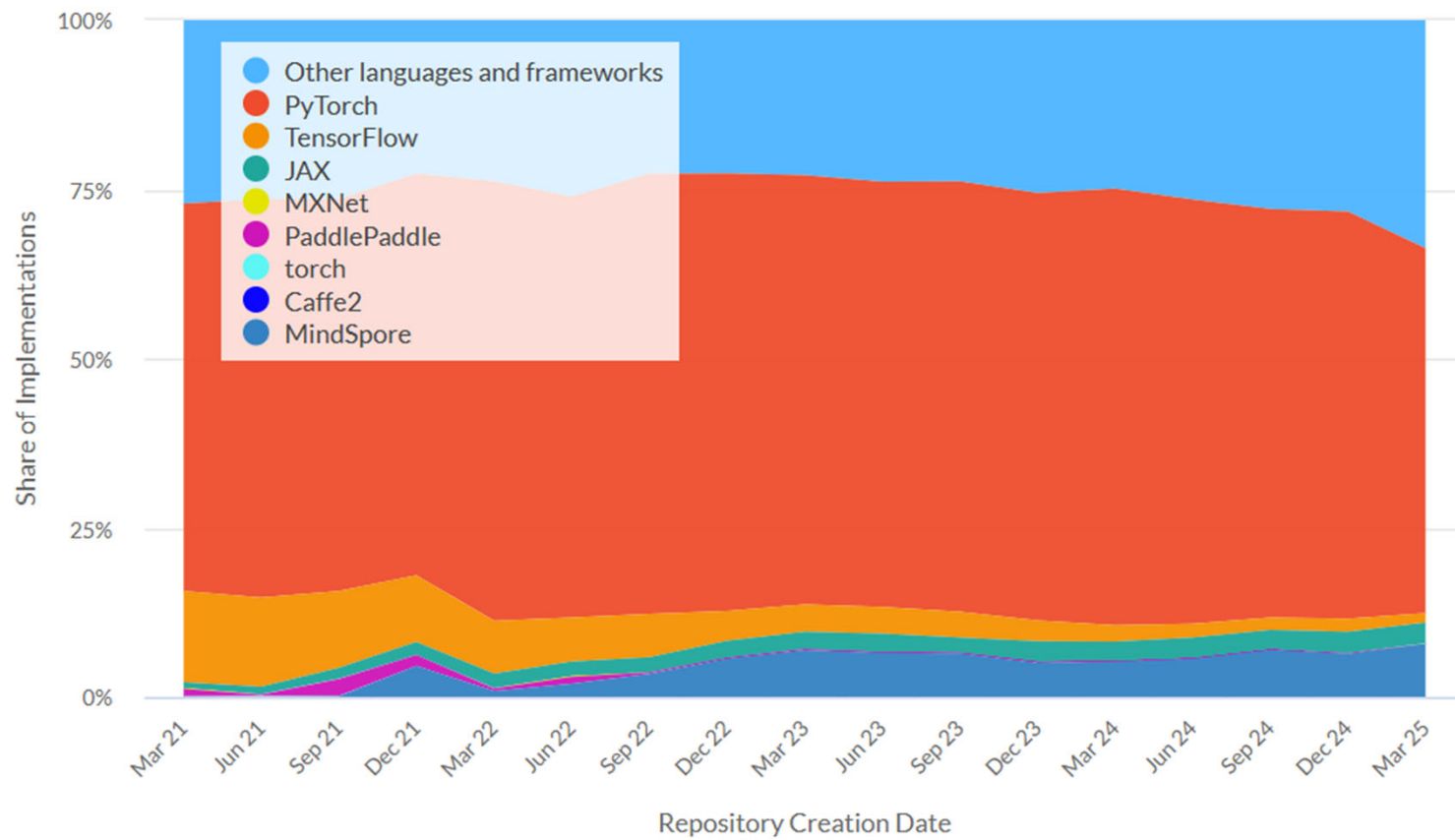


Why pytorch?



How do train a model?

$$\theta^* = \arg \min_{\theta} \sum_{(x,y) \in D} \mathcal{L}(f_{\theta}(x), y)$$

gradient descent

dataset

loss

neural network

PyTorch does all of these!

What is PyTorch?

Python library for:

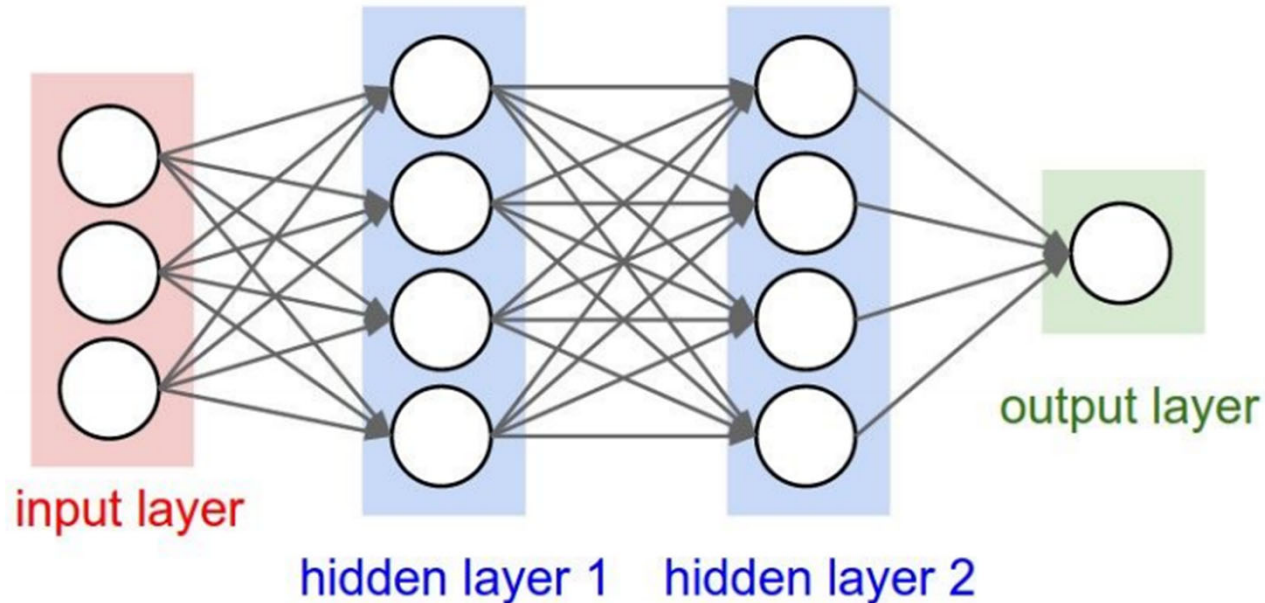
- Defining neural networks
- Automating computing gradients
- And more! (datasets, optimizers, GPUs, etc.)

$$\theta^* = \arg \min_{\theta} \sum_{(x,y) \in D} \mathcal{L}(f_{\theta}(x), y)$$

Diagram illustrating the optimization equation for finding the optimal parameters θ^* :

- $\arg \min_{\theta}$ is associated with **gradient descent**.
- $\sum_{(x,y) \in D}$ is associated with **dataset**.
- $\mathcal{L}$ is associated with **loss**.
- $f_{\theta}(x)$ is associated with **neural network**.

How does PyTorch work?



You define:

$$h_1 = \sigma(W_1 x) \quad h_2 = \sigma(W_2 h_1) \quad y = \sigma(W_3 h_2)$$

PyTorch computes:

$$\frac{\partial y}{\partial W_1} = \frac{\partial y}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial W_1} \quad \frac{\partial y}{\partial W_2} = \frac{\partial y}{\partial h_2} \frac{\partial h_2}{\partial W_2} \quad \frac{\partial y}{\partial W_3}$$



- Fast CPU implementations
- **CPU-only**
- **No autodiff**
- Imperative



- Fast CPU implementations
- **Allows GPU**
- **Supports autodiff**
- Imperative

Other features include:

- Datasets and dataloading
- Common neural network operations
- Built-in optimizers (Adam, SGD, ...)

The Basics



```
arr_a = [1, 3, 4, 5, 9]
arr_b = [9, 5, 7, 2, 5]

# Element-wise operations
list_sum = [a + b for a, b in zip(list_a, list_b)]
list_prod = [a * b for a, b in zip(list_a, list_b)]
list_doubled = [2 * a for a in list_a]

# Indexing
value = list_a[3]
list_slice = list_a[2:3]

arr_idx = [3, 2, 1]
arr_indexed = [arr_a[i] for i in arr_idx]
```



```
import numpy as np

arr_a = np.array([1, 3, 4, 5, 9])
arr_b = np.array([9, 5, 7, 2, 5])

# Element-wise operations
arr_sum = a + b
arr_prod = a * b
arr_doubled = 2 * a

# Indexing
value = arr_a[3]
arr_slice = arr_a[2:3]

arr_idx = np.array([3, 2, 1])
arr_indexed = arr_a[arr_idx]
```



```
import torch

tensor_a = torch.tensor([1, 3, 4, 5, 9])
tensor_b = torch.tensor([9, 5, 7, 2, 5])

# Element-wise operations
tensor_sum = tensor_a + tensor_b
tensor_prod = tensor_a * tensor_b
tensor_doubled = 2 * tensor_a

# Indexing
value = tensor_a[3]
tensor_slice = tensor_a[2:3]

tensor_idx = torch.tensor([3, 2, 1])
tensor_indexed = tensor_a[tensor_idx]
```

100x faster!

Multidimensional Indexing



A

Axis 0 ↓	Axis 1 →				
	32	27	5	54	1
	99	4	23	3	57
	76	42	34	82	5

`A.shape == (3, 5)`

Multidimensional Indexing



Axis 1

Axis 0

32	27	5	54	1
99	4	23	3	57
76	42	34	82	5

$A[0, 3]$

Multidimensional Indexing



Axis 1

Axis 0

32	27	5	54	1
99	4	23	3	57
76	42	34	82	5

$A[:, 3]$

Multidimensional Indexing



Diagram illustrating a 2D array with axes labeled:

- Axis 0 (vertical arrow pointing down)
- Axis 1 (horizontal arrow pointing right)

32	27	5	54	1
99	4	23	3	57
76	42	34	82	5

$A[0, :]$

Multidimensional Indexing



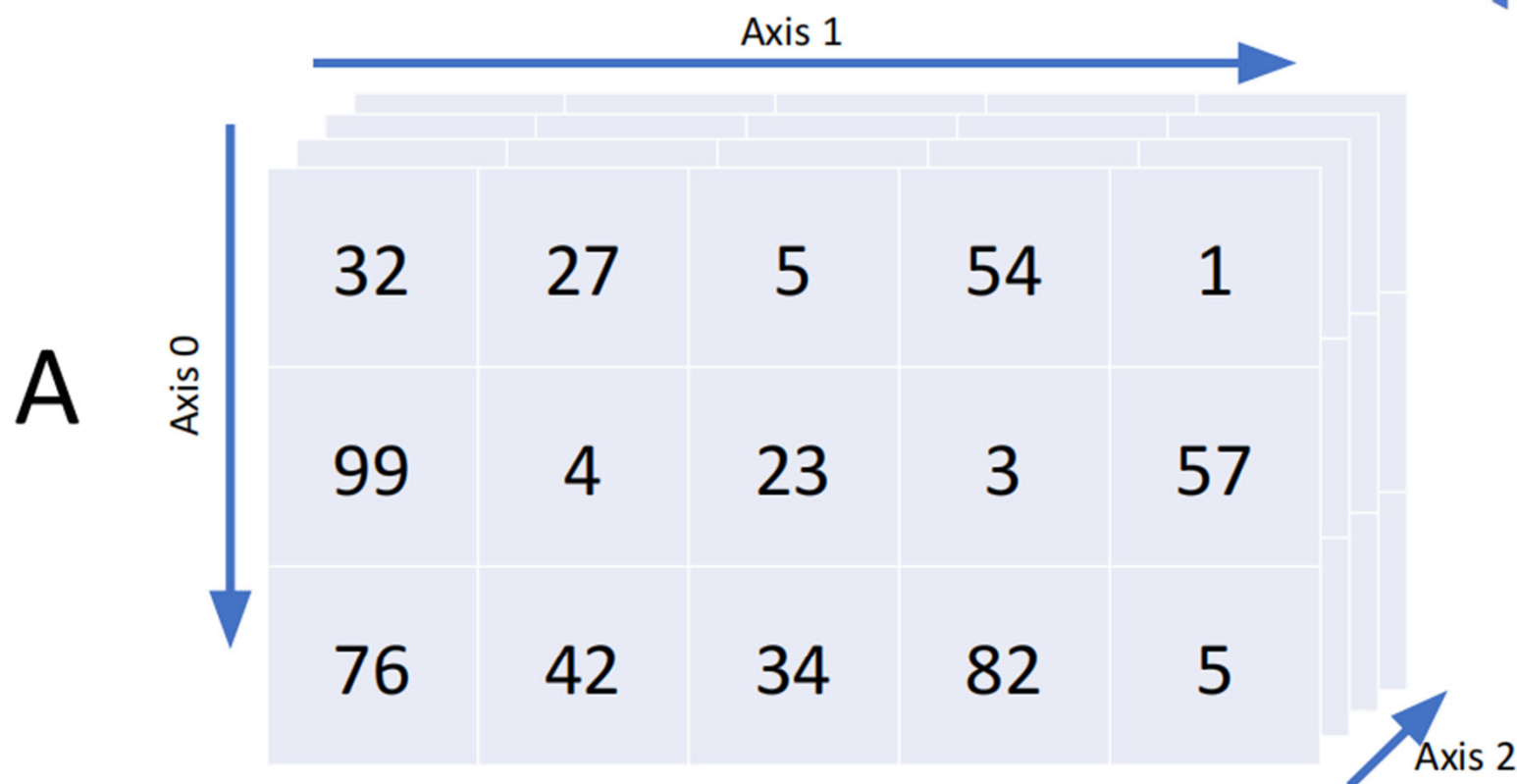
Axis 1

Axis 0

32	27	5	54	1
99	4	23	3	57
76	42	34	82	5

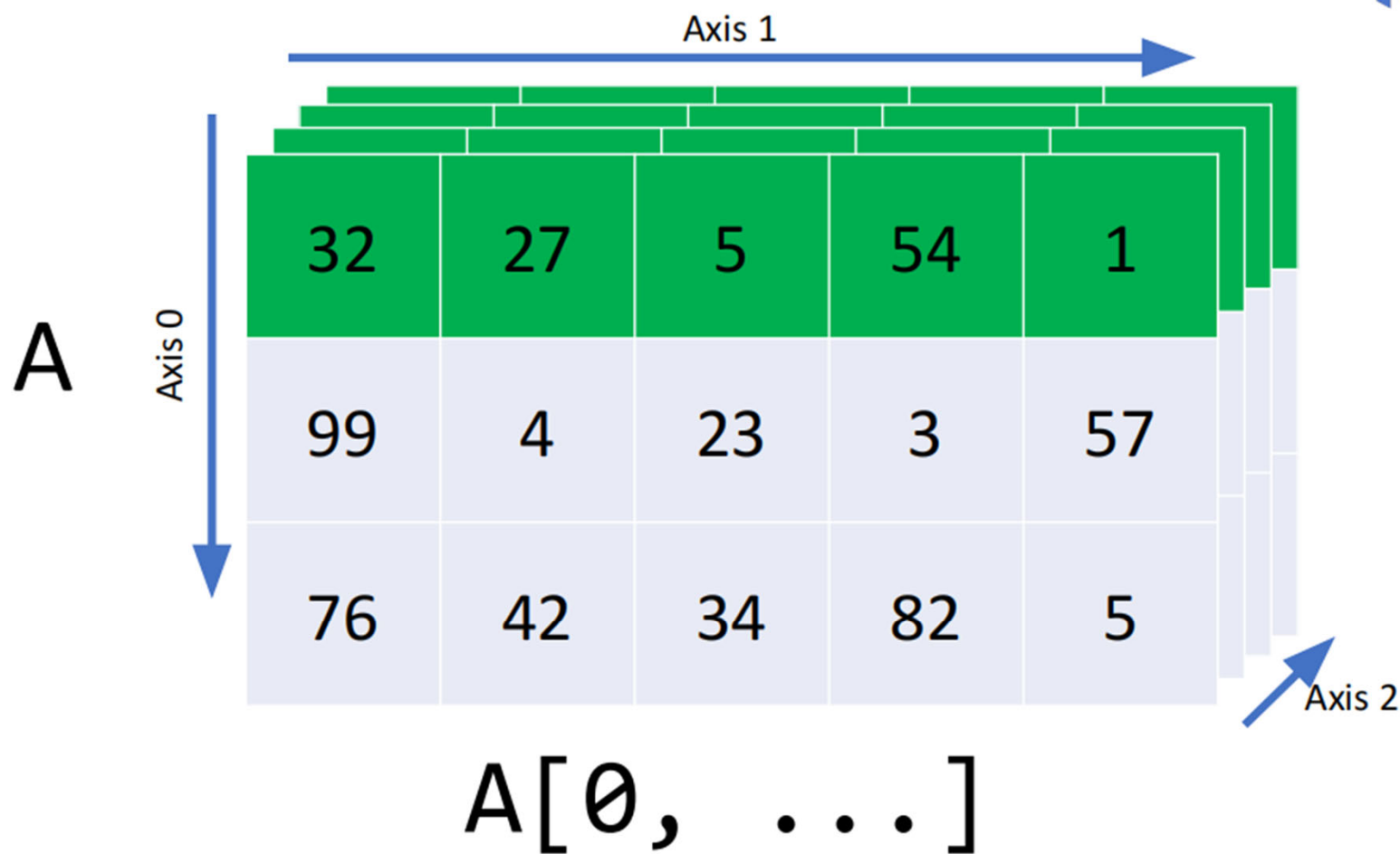
$A[0, 2:4]$

Multidimensional Indexing

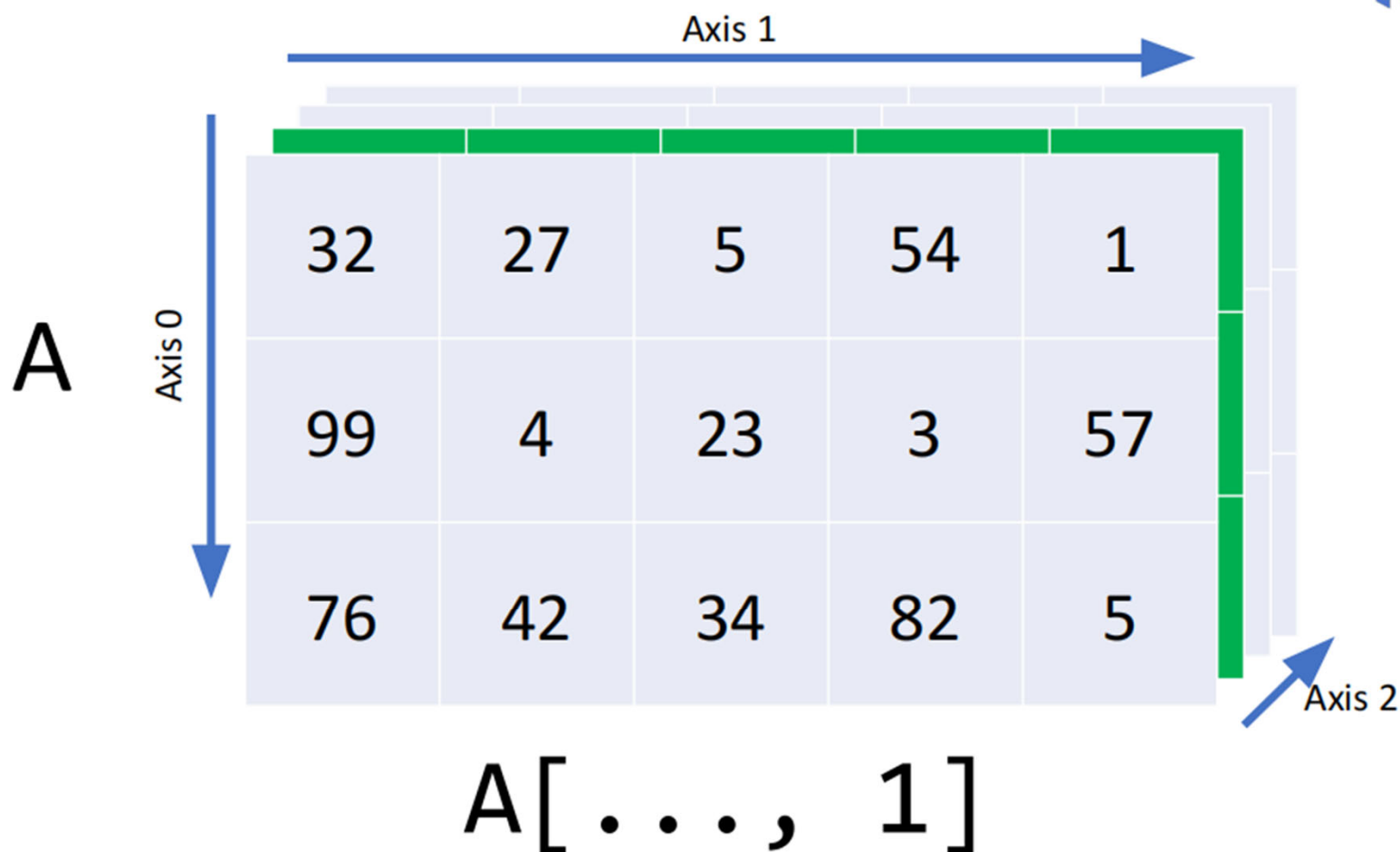


`A.shape == (3, 5, 4)`

Multidimensional Indexing



Multidimensional Indexing



Shape Operations



```
A = np.random.normal(size=(10, 15))

# Indexing with newaxis/None
# adds an axis with size 1
A[np.newaxis] # -> shape (1, 10, 15)

# Squeeze removes a axis with size 1
A[np.newaxis].squeeze(0) # -> shape (10, 15)

# Transpose switches out axes.
A.transpose((1, 0)) # -> shape (15, 10)

# !!! BE CAREFUL WITH RESHAPE !!!
A.reshape(15, 10) # -> shape (15, 10)
A.reshape(3, 25, -1) # -> shape (3, 25, 2)
```



```
A = torch.randn((10, 15))

# Indexing with None
# adds an axis with size 1
A[None] # -> shape (1, 10, 15)

# Squeeze removes a axis with size 1
A[None].squeeze(0) # -> shape (10, 15)

# Permute switches out axes.
A.permute((1, 0)) # -> shape (15, 10)

# !!! BE CAREFUL WITH VIEW !!!
A.view(15, 10) # -> shape (15, 10)
A.view(3, 25, -1) # -> shape (3, 25, 2)
```

Device Management

- Numpy: all arrays live on the CPU's RAM
- Torch: tensors can either live on CPU or GPU memory
 - Move to GPU with `.to("cuda")` / `.cuda()`
 - Move to CPU with `.to("cpu")` / `.cpu()`

**YOU CANNOT PERFORM OPERATIONS BETWEEN
TENSORS ON DIFFERENT DEVICES!**

In [72]:

```
# Set device type  
device = "cuda" if torch.cuda.is_available() else "cpu"  
device
```

Out[72]: 'cuda'

Put tensor to device

```
# Create tensor (default on CPU)  
tensor = torch.tensor([1, 2, 3])  
  
# Tensor not on GPU  
print(tensor, tensor.device)  
  
# Move tensor to GPU (if available)  
tensor_on_gpu = tensor.to(device)  
tensor_on_gpu
```

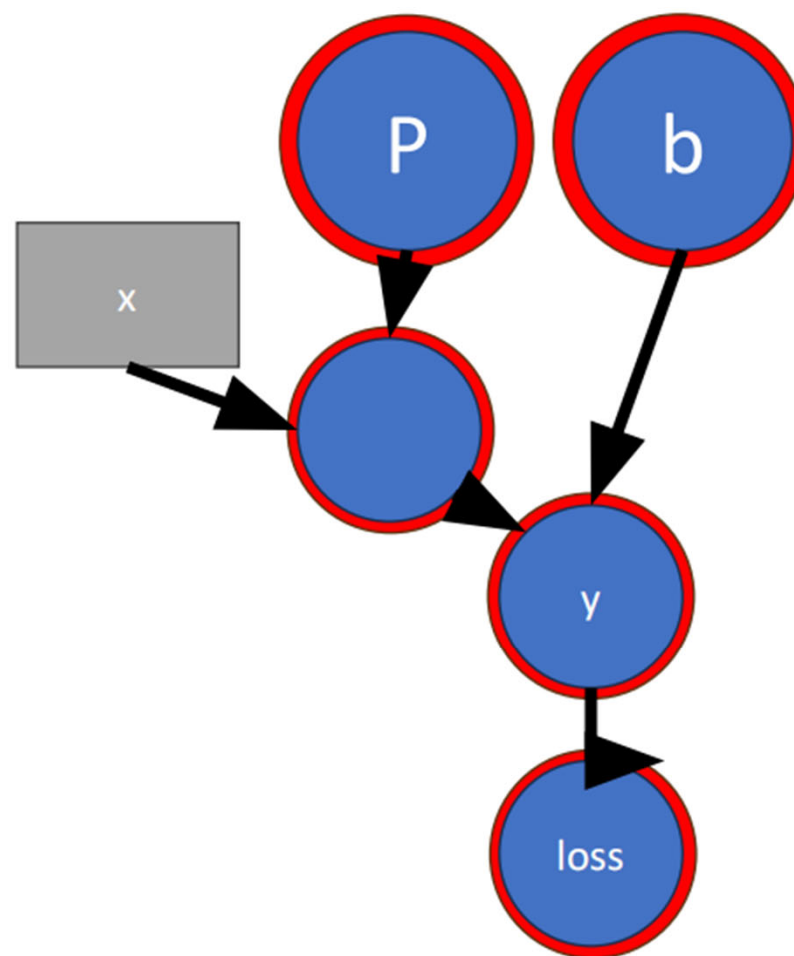
```
[ ] device = torch.device("cuda")
x = torch.zeros((2, 3))
y = torch.ones((2, 3), device=device)
z = x + y
```

```
-----
RuntimeError                                Traceback (most recent call last)
<ipython-input-71-565d7b7035e6> in <module>
      2 x = torch.zeros((2, 3))
      3 y = torch.ones((2, 3), device=device)
----> 4 z = x + y
```

```
RuntimeError: Expected all tensors to be on the same device, but found at least two
devices, cuda:0 and cpu!
```

Computing Gradients

```
P = torch.randn((1024, 1024))
print(P.requires_grad) # -> False
P = torch.randn((1024, 1024), requires_grad=True)
b = torch.randn((1024,)), requires_grad=True)
print(P.grad) # -> None
```

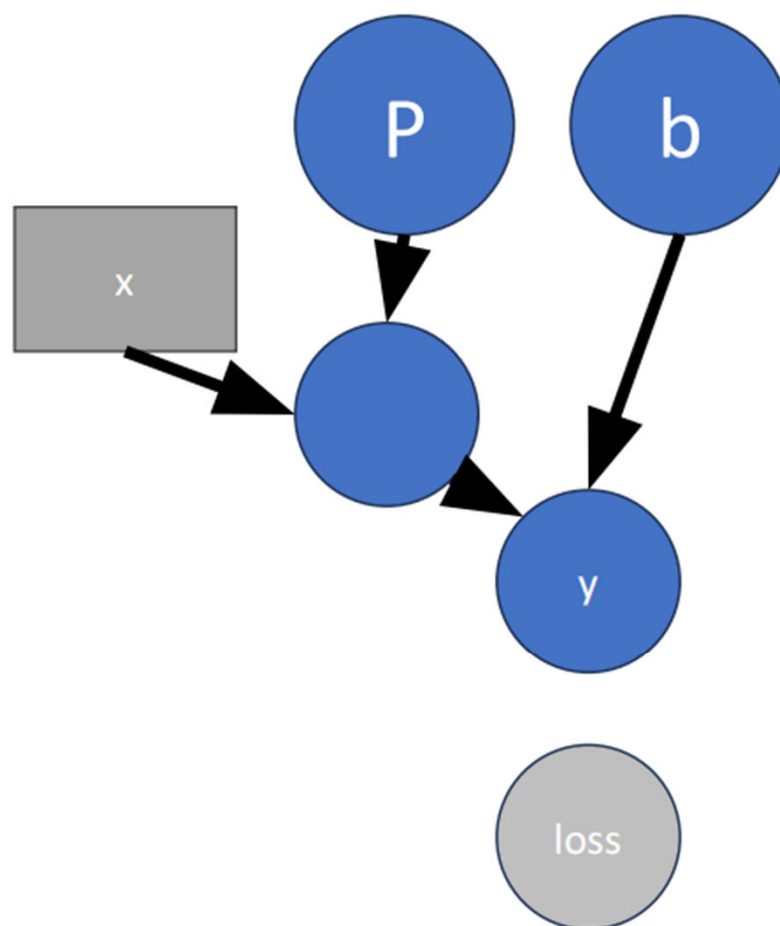


Computing Gradients

```
P = torch.randn((1024, 1024))
print(P.requires_grad) # -> False
P = torch.randn((1024, 1024), requires_grad=True)
b = torch.randn((1024, ), requires_grad=True)
print(P.grad) # -> None

x = torch.randn((32, 1024))
y = torch.nn.relu(x @ P + b)

target = 3
loss = torch.mean((y - target) ** 2).detach()
```



Example

- create a simple tensor that requires gradient tracking and compute the result of $f(x) = x^2 + 2x + 1$. Then trigger backpropagation.

```
x = torch.tensor([2.0], requires_grad=True)
y = x**2 + 2*x + 1
y.backward()
print(f"Gradient of y with respect to x: {x.grad}")
```

Quiz time

- Which method is used to convert a PyTorch tensor to a NumPy array?
- A) `to_numpy()`
 - B) `numpy()`
 - C) `as_numpy()`
 - D) `detach().numpy()`

Quiz time

- What is the correct way to move a tensor to GPU if available?
 - A) `tensor = tensor.cuda()`
 - B) `tensor = tensor.to('gpu')`
 - C) `tensor = tensor.to(torch.device('cuda' if torch.cuda.is_available() else 'cpu'))`
 - D) `tensor = tensor.gpu()`

Quiz time

- Which of the following creates a 1D tensor from a Python list?

A) `torch.Tensor([1, 2, 3])`

B) `torch.array([1, 2, 3])`

C) `torch.list_to_tensor([1, 2, 3])`

D) `torch.from_list([1, 2, 3])`

Quiz time

- What is the result of the following code?

```
a = torch.tensor([1, 2, 3])
```

```
b = torch.tensor([4, 5, 6])
```

```
print(torch.cat((a, b)))
```

Quiz time

- Write 3-4 lines of code to create a zero tensor of size 3x3, then change the values of the middle row to 1s.

```
z = torch.zeros(3, 3)
z[1, :] = 1 # Sets middle row to all 1s
```

Converting Numpy / PyTorch

Numpy -> PyTorch:

```
torch.from_numpy(numpy_array).float()
```

PyTorch -> Numpy:

- (If requires_grad) Get a copy without graph with `.detach()`
- (If on GPU) Move to CPU with `.to("cpu")/.cpu()`
- Convert to numpy with `.numpy`

All together:

```
torch_tensor.detach().cpu().numpy()
```

Neural network training

Import torch

```
from torch.utils.data import DataLoader, TensorDataset
```

```
X = torch.randn(100, 10) # 100 samples, 10 features
```

```
y = torch.randint(0, 2, (100,)) # Binary classification
```

```
# 3. Create dataset and dataloader
```

```
dataset = TensorDataset(X, y)
```

```
dataloader = DataLoader(dataset, batch_size=16, shuffle=True)
```

Neural network training

```
import torch.nn as nn
```

1. Define the neural network

```
class SimpleNet(nn.Module):
```

```
    def __init__(self, input_size, output_size):
```

```
        super(SimpleNet, self).__init__()
```

```
        self.layer1 = nn.Linear(input_size, output_size)
```

```
        self.relu = nn.ReLU()
```

```
    def forward(self, x):
```

```
        x = self.layer1(x)
```

```
        x = self.relu(x)
```

```
        return x
```

```
model = SimpleNet(input_size=3, output_size=2)
```

```
x = torch.Tensor([1, 2, 3])
```

```
print(model(x))
```

Neural network training

```
criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(model.parameters(), lr=0.01)
# 5. Training loop
num_epochs = 5
for epoch in range(num_epochs):
    for inputs, labels in dataloader:
        # Forward pass
        outputs = model(inputs)
        loss = criterion(outputs, labels)
        # Backward pass and optimization
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    print(f'Epoch [{epoch+1}/{num_epochs}], Loss: {loss.item():.4f}')
```

Training Loop

REMEMBER THIS!

```
net = (...).to("cuda")
dataset = ...
dataloader = ..
optimizer = ...
loss_fn = ..
for epoch in range(num_epochs):
    # Training..
    net.train()
    for data, target in dataloader:
        data = torch.from_numpy(data).float().cuda()
        target = torch.from_numpy(target).float().cuda()

        prediction = net(data)
        loss = loss_fn(prediction, target)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    net.eval()
    # Do evaluation..
```